ECE551: Digital System Design and Synthesis

Fall 2026

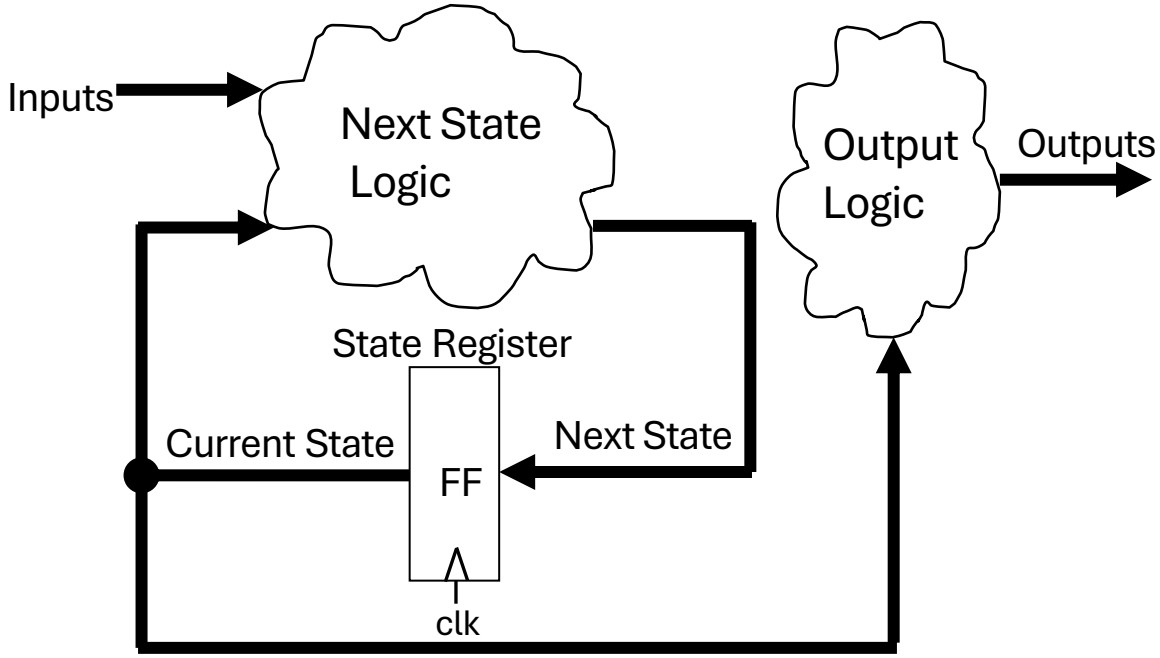
Finite State Machines

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

FSM Review

- Often used to generate control signals
- Reacts to inputs (including clock signal)
- Can perform multi-cycle operations
- Combinational and sequential logic

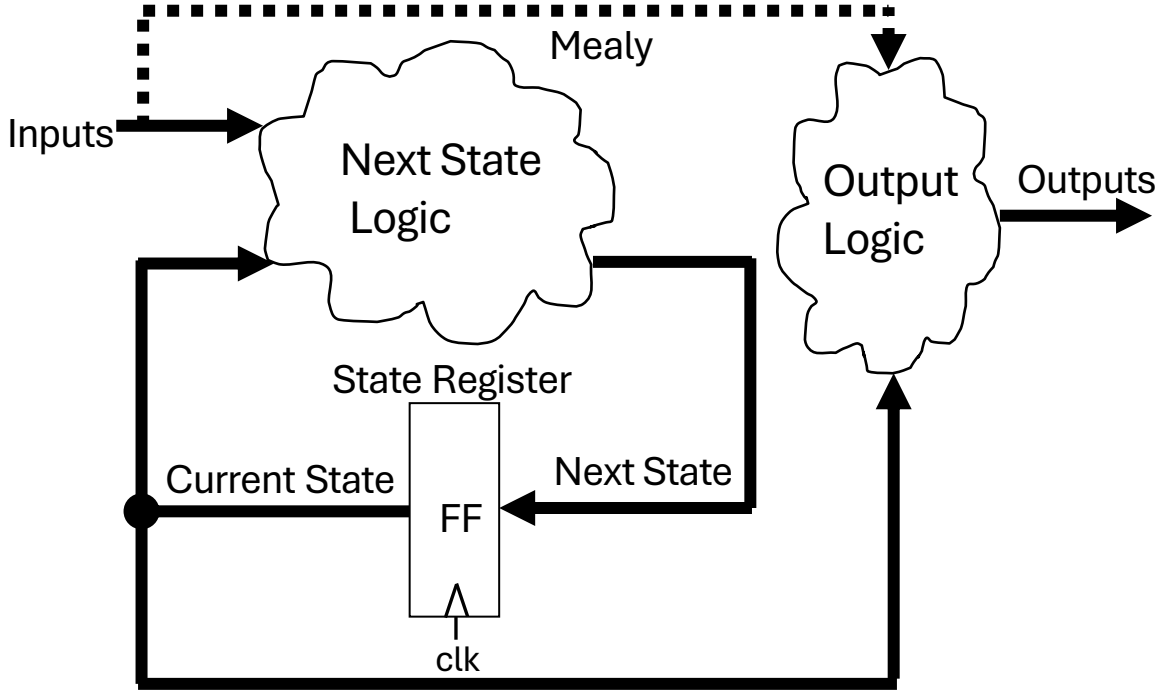
- Examples of FSMs:
  - Counter
  - Vending machine
  - Traffic light controller
  - Bus Controller
  - Control unit of serial protocol (like RS232, I2C or SPI)



FSM Review

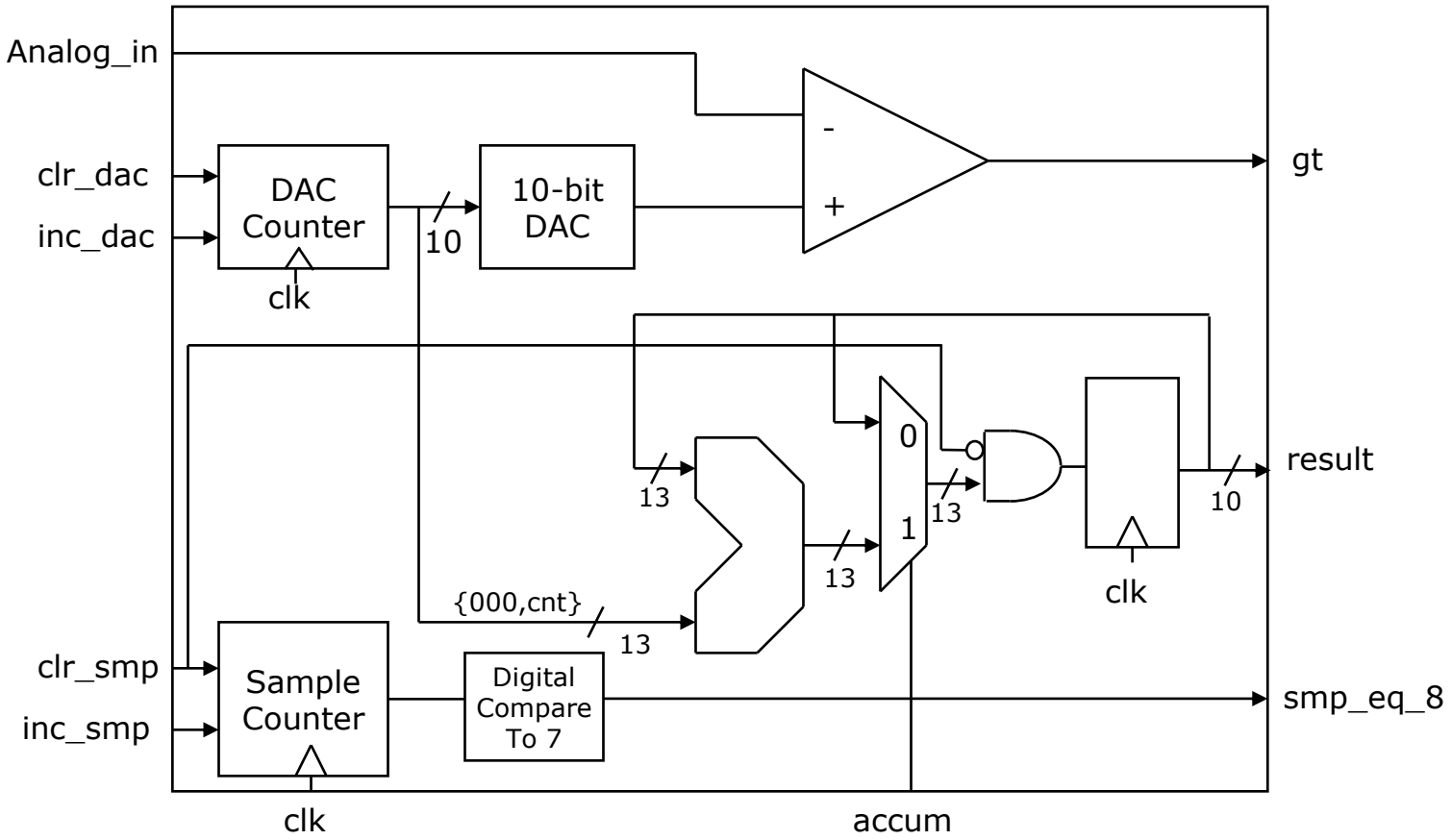
- Moore
  - Output depends only on current state
  - Outputs are synchronous (but not necessarily glitch free)

- Mealy
  - Output depends on current state and inputs
  - Outputs can be asynchronous
    - Change with changes on the inputs
  - Outputs can be synchronous
    - Register the outputs
    - Outputs delayed by one cycle



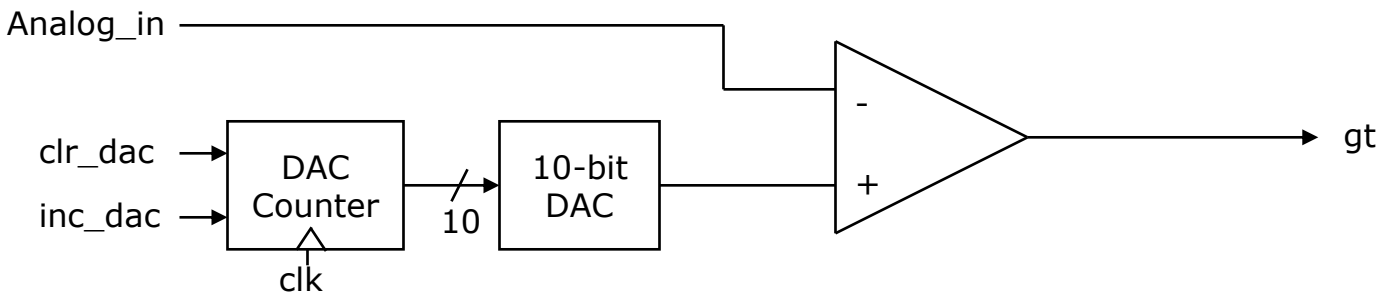
FSM example

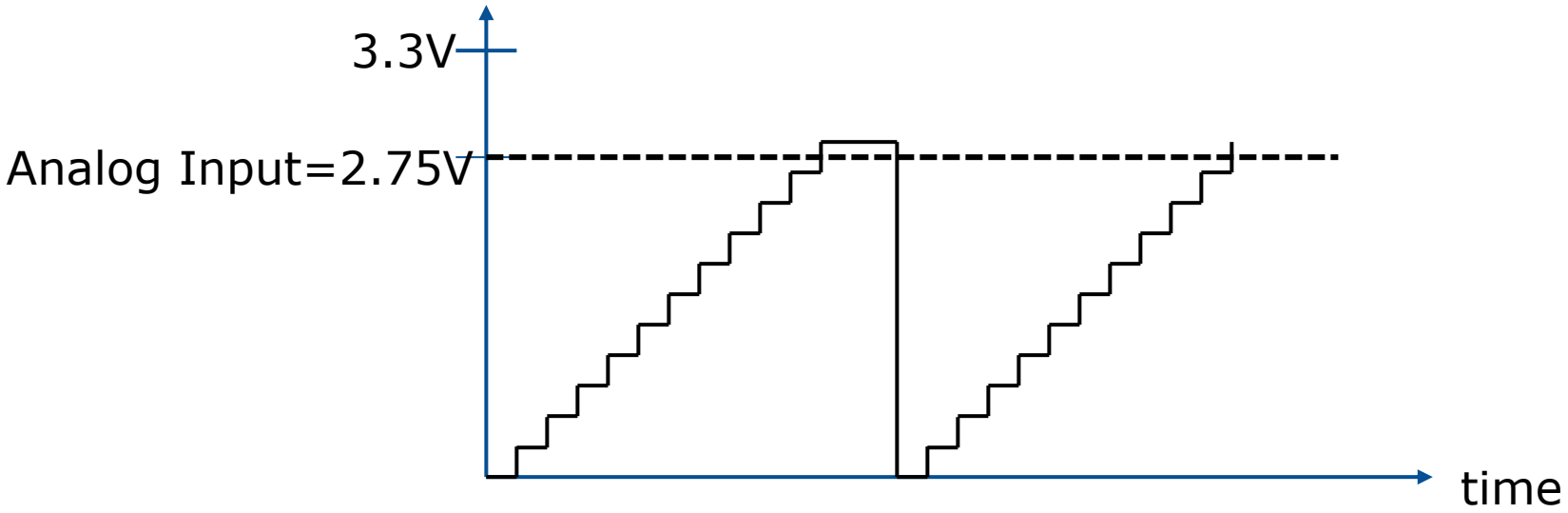
- Given the datapath for a single slope A2D converter, draw a state (bubble) diagram for an FSM that can control it.
- It should run the converter for 8 times and accumulate the 8 results in a 13-bit register. The upper 10-bits of which are used as the result.



FSM example

- Single slope A2D converter:

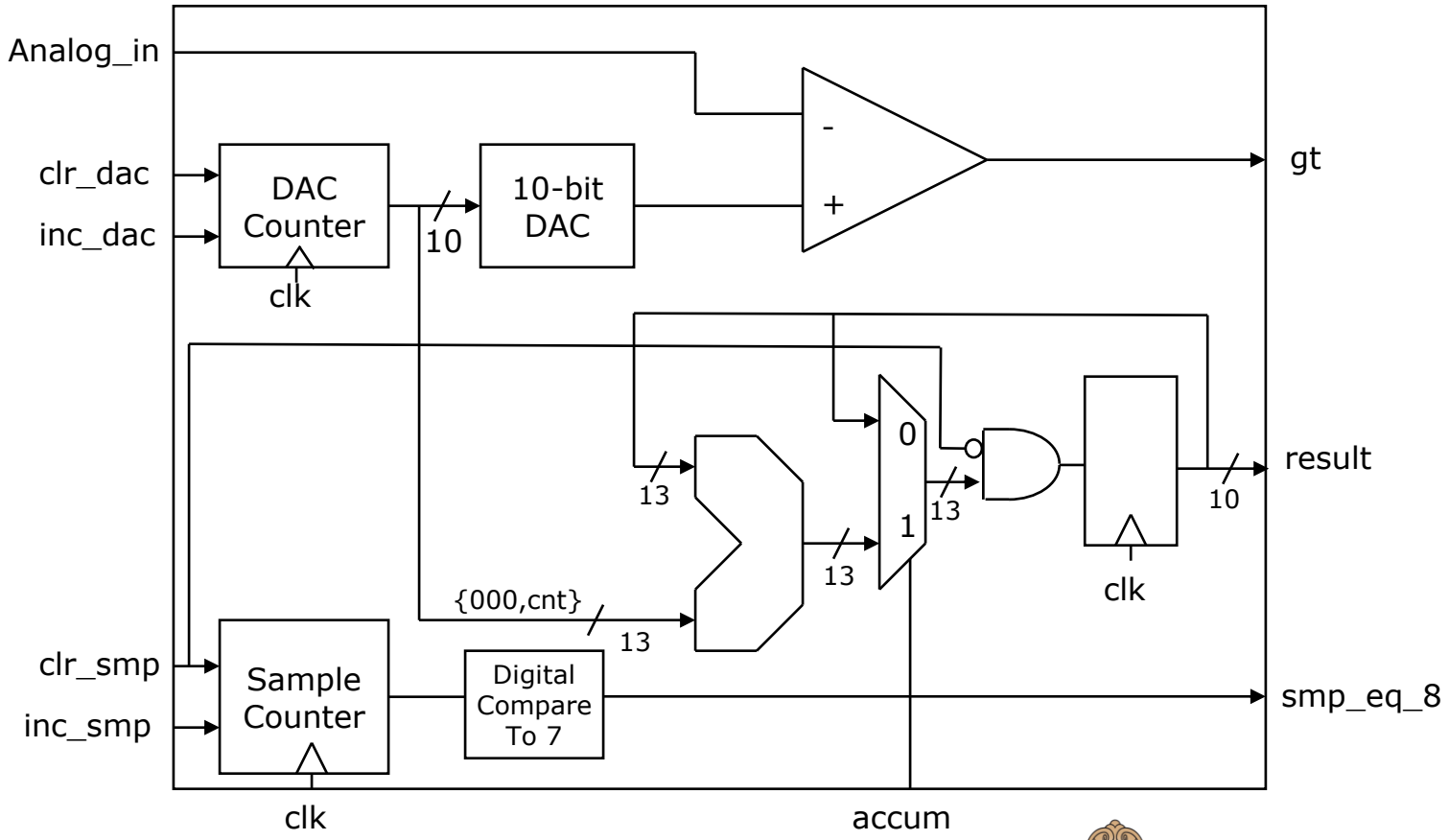




FSM example

- What are the state machine inputs & outputs?
  - All control inputs in the datapath are state machine outputs

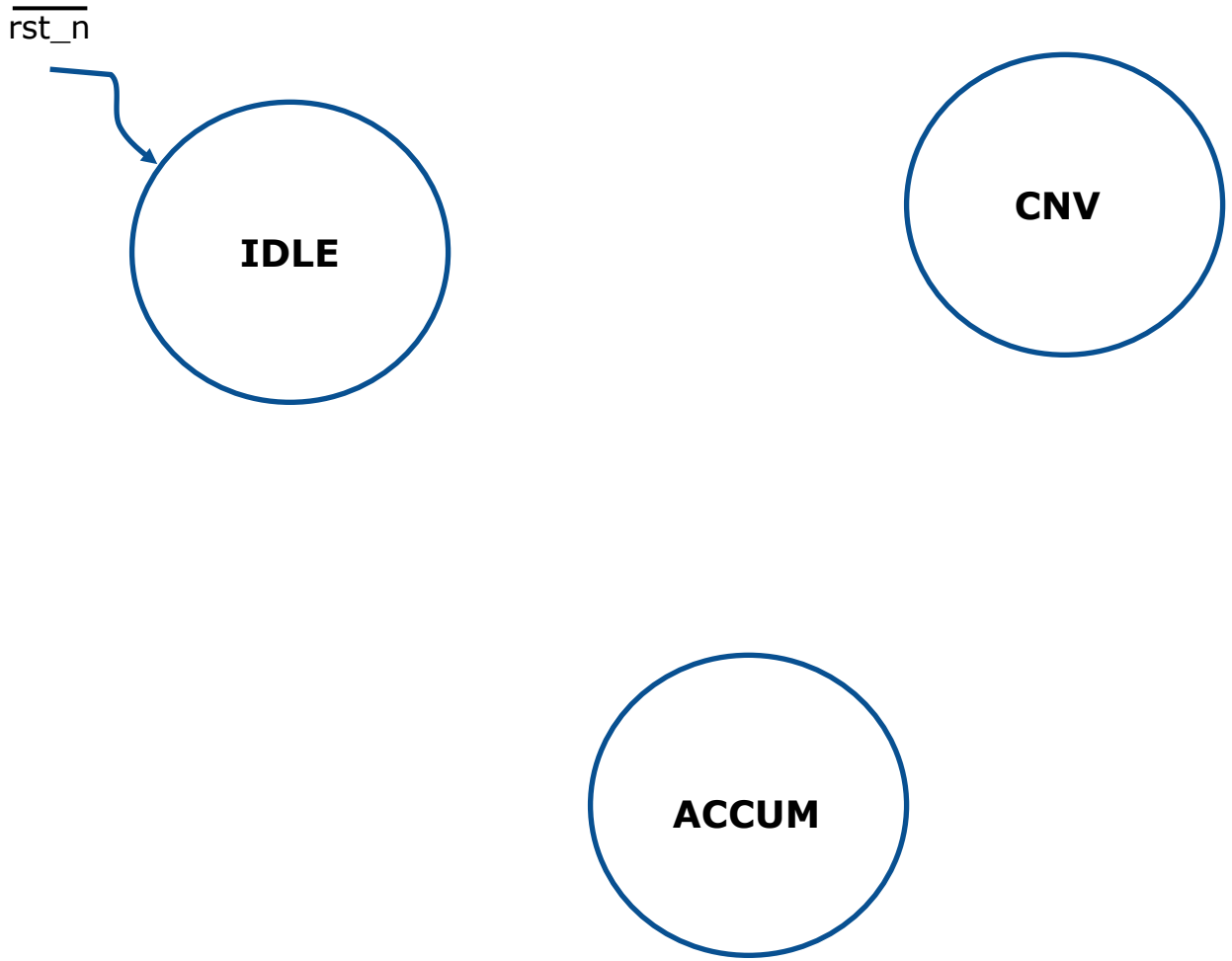
Inputs	Outputs
gt	clr_dac
smp_eq_8	inc_dac
strt_cnv	clr_smp
clk	inc_smp
rst_n	accum
	cnv_cmplt



Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

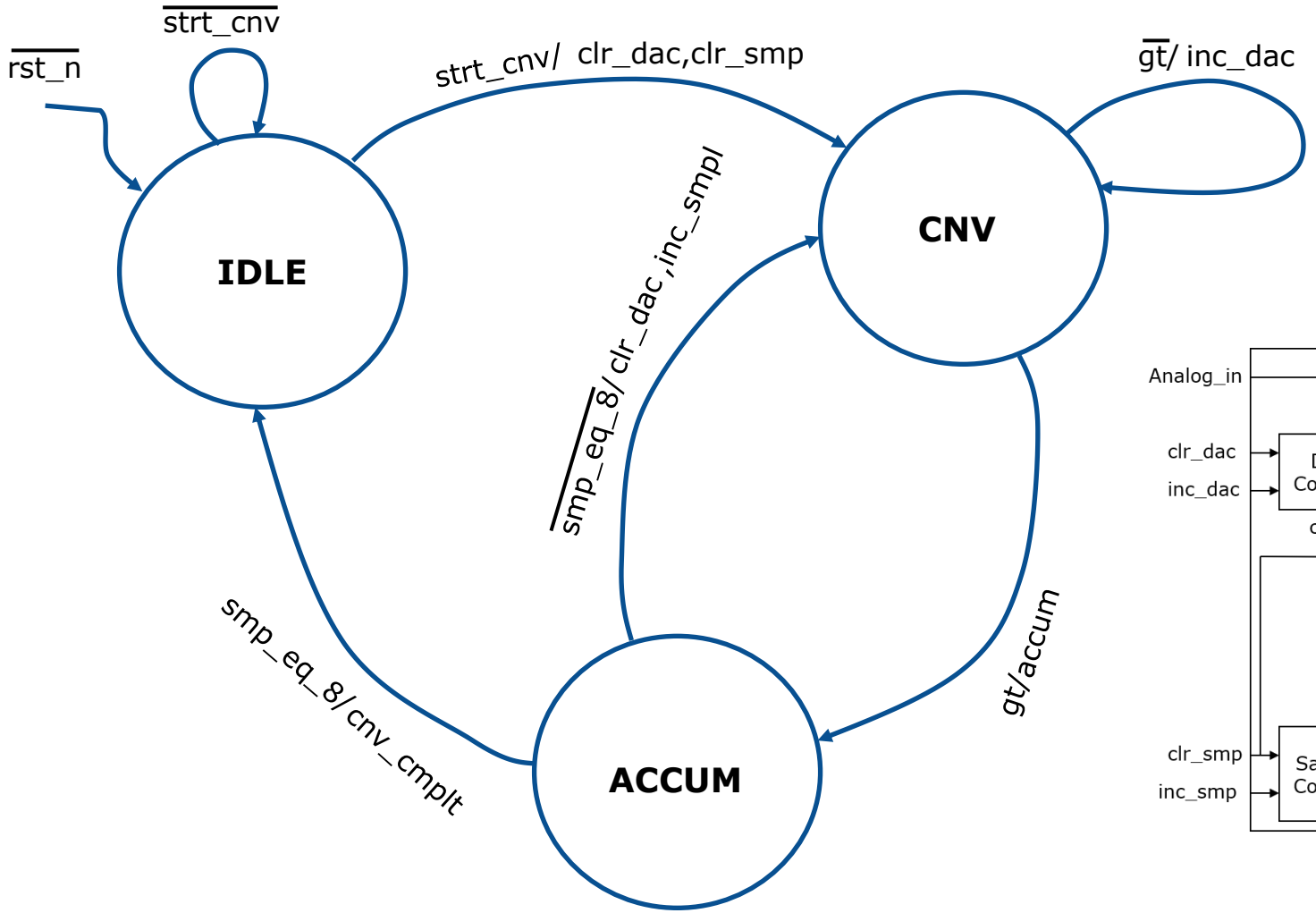
FSM example

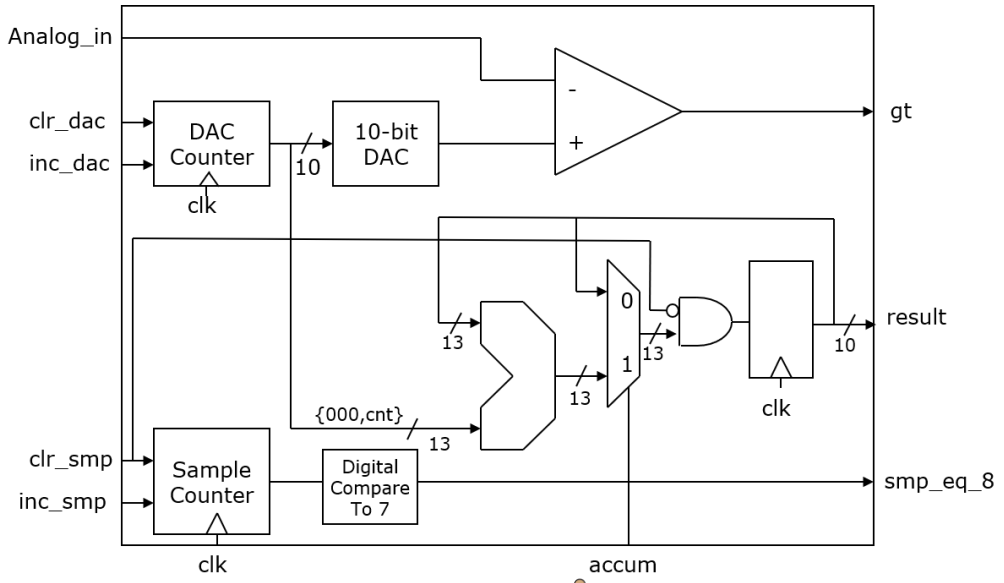
- State/Bubble diagram:



FSM example

- State/Bubble diagram:





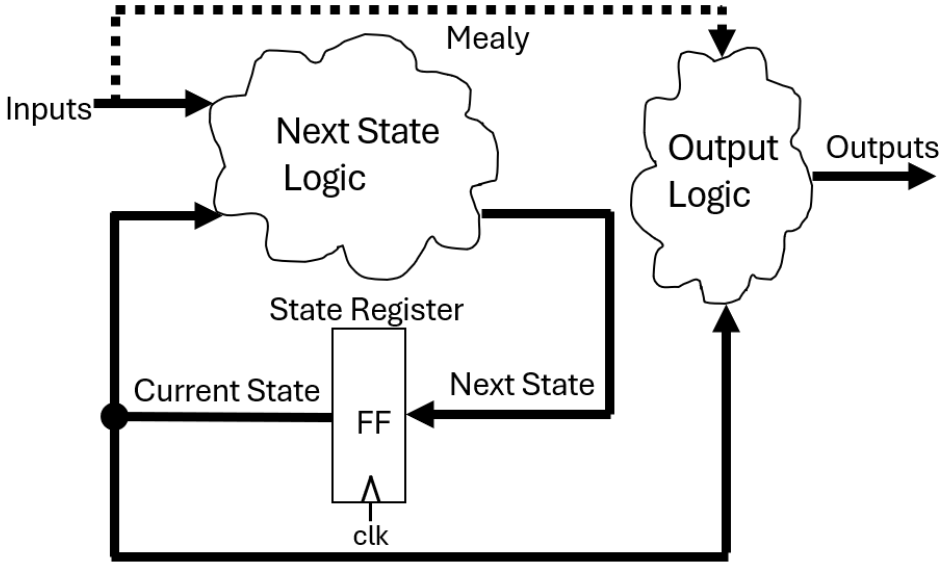
Coding FSM

- What is the best way to code FSM?
  - Best to separate combinational (blocking) from sequential (non-blocking)

    → Combinational logic here is more complex and not guaranteed to evaluate in 1 sim cycle

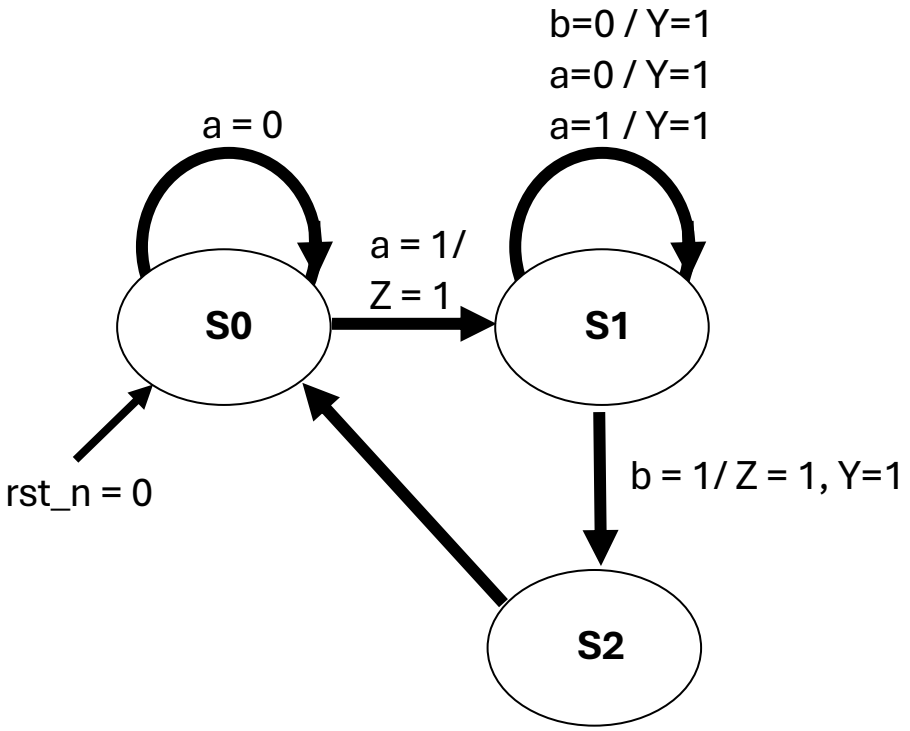
  - Output logic and state transition logic can be coded in the same **always** block since they have the same inputs

  - Output logic and state transition logic are ideally suited for a **case** statement



Simple Example

- Let's code this simple example:



Outputs **Y** and **Z** are 0, unless specified otherwise.

```
module fsm(clk,rst_n,a,b,Y,Z);
  input clk,rst_n,a,b;
  output reg Y,Z;

  typedef enum reg [1:0] {S0, S1,
                          S2} state_t;
  state_t state,nxt_state;

  always_ff @(posedge clk,
              negedge rst_n)
    if (!rst_n)
      state <= S0;
    else
      state <= nxt_state;
```

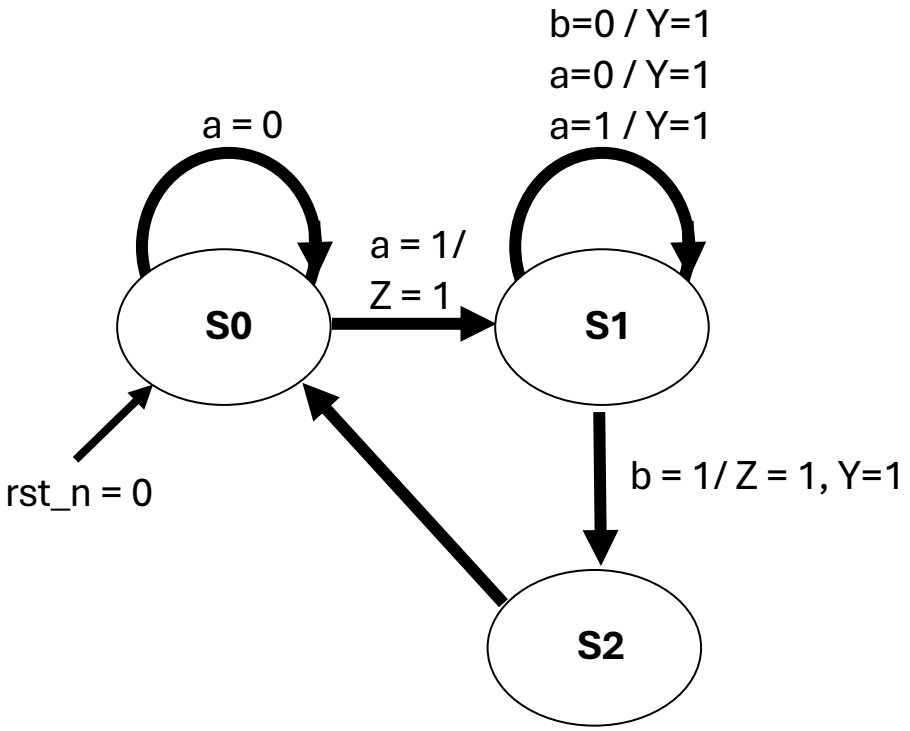
```
always_comb
  case (state)
    S0 : if (a) begin
           nxt_state = S1;
           Z = 1; end
         else
           nxt_state = S0;
    S1 : begin
           Y=1;
           if (b) begin
              nxt_state = S2;
              Z=1;
           end else
              nxt_state = S1;
           end
    S2 : nxt_state = S0;
  endcase
endmodule
```

What problems do we have here?

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN-MADISON

Simple Example

- Let's code this simple example:



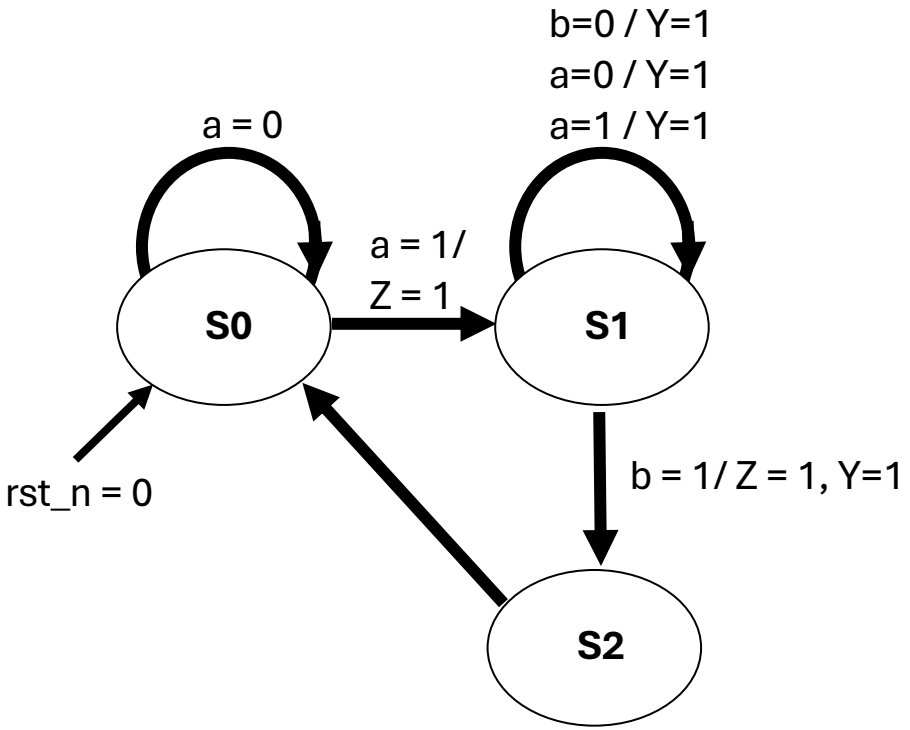
Outputs **Y** and **Z** are 0, unless specified otherwise.

```
always_comb
  nxt_state = S0;
  Z=0;
  Y=0;
  case (state)
    S0 : if (a) begin
           nxt_state = S1;
           Z = 1; end
         else
           nxt_state = S0;
```

```
S1 : begin
           Y=1;
           if (b) begin
              nxt_state = S2;
              Z=1;
           end else
              nxt_state = S1;
           end
    default : nxt_state = S0;
  endcase
endmodule
```

Simple Example

- Let's code this simple example:




```
always_comb
  nxt_state = S0;
  Z=0;
  Y=0;
  case (state)
    S0 : if (a) begin
           nxt_state = S1;
           Z = 1; end
         else
           nxt_state = S0;
```

```
S1 : begin
       Y=1;
       if (b) begin
         nxt_state = S2;
         Z=1;
       end else
         nxt_state = S1;
       end
   default : nxt_state = S0;
  endcase
endmodule
```

Outputs **Y** and **Z** are 0, unless specified otherwise.

Defaulting of assignments and having a default to the case is highly recommended!

FSM Coding Guidelines

1. Keep state assignment in separate **always_ff** block using non-blocking "<=" assignment
2. Code state transition logic and output logic together in an **always_comb** block using blocking assignments
3. Assign default values to all outputs, and the **nxt_state** registers. This helps avoid unintended latches
4. Remember to have a **default** to the **case** statement.
   - Default should be (if possible) a state that transitions to the same state as reset would take the SM to.
   - Avoids latches
   - Makes design more robust to spurious electrical/cosmic events.